

Chapter 13 .딥러닝 모델 저장/복원/조기종료

- 케라스 딥러닝 모델 저장 및 복원
- 조기 종료 콜백

* keras 모델 저장 / 복원

: 훈련 모델 및 가중치 저장 메서드 제공

모델 저장

```
model.save('D:\Wkeras_model_save\mywine_clf.h5')
```

: 훈련이 끝난 전체 모델의 가중치, 옵티마이저 등,
전체 구조 모두 저장

```
model.save_weights('D:\Wkeras_model_save\mywine_clf.h5')
```

: 훈련이 끝난 모델의 가중치 저장

모델 복원

```
from keras.models import load_model  
new_model =  
load_model('D:\Wkeras_model_save\mywine_clf.h5')
```

: 저장된 모델 가중치,옵티마이저,구조 모두 복원

```
model.load_weights('D:\Wkeras_model_save\mywine_clf.h5')
```

: 저장된 모델의 가중치 복원해 불러오기
단, save_weights() 메서드로 저장했던 모델과
정확히 동일 모델 구조일때 가능

또한, load_weights()메서드로 가중치만 load 한 경우
동일 모델 설계 후 사용해야 함



* keras 전체 모델 저장 / 복원

```
import numpy as np
import pandas as pd
```

```
winedf = pd.read_csv('wine.csv', header=None)
print(winedf.head())
```

```
winedata = winedf.values
x_data = winedata[:,0:12] # 훈련 특성데이터 추출
y_data = winedata[:,12] # 타겟 데이터 추출
```

```
# 훈련셋, 검증셋 분리
from sklearn.model_selection import train_test_split
x_train, x_val, y_train, y_val = train_test_split(x_data, y_data, test_size=0.2, random_state=42)
```

0	주석산 농도	7	밀도
1	아세트산 농도	8	pH
2	구연산 농도	9	황산칼륨 농도
3	잔류 당분 농도	10	알코올 도수
4	염화나트륨 농도	11	와인의맛 (등급)
5	유리 아황산 농도	12	class (1:레드와인, 0:화이트와인)
6	총 아황산 농도		

모델 복원 후
검증데이터로 모델 평가

모델 설계

```
from tensorflow.keras.models import Sequential  
from tensorflow.keras.layers import Dense
```

```
model = Sequential()  
model.add(Dense(30, input_dim=12, activation='relu'))  
model.add(Dense(12, activation='relu'))  
model.add(Dense(8, activation='relu'))  
model.add(Dense(1, activation='sigmoid'))
```

모델 컴파일

```
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
```

모델 훈련

```
model.fit(x_train, y_train, epochs=100, batch_size=200)
```

모델 평가

```
print('accuracy : %.4f ' %model.evaluate(x_train,y_train)[1])
```

accuracy : 0.9631

모델 저장 == > 오류시 h5py 모듈 2.10.0 버전 설치

모델 저장 경로 : 영문 경로로 지정

```
model.save('D:\Wkeras_model_save\mywine_clf.h5')
```

지정 디렉토리에
~.h5 파일로 저장되 있음.
h5 : HDF5포맷 저장

기존 훈련 모델 삭제

```
del model
```

저장된 모델 새로 불러옴

```
from keras.models import load_model
```

```
new_model = load_model('D:\Wkeras_model_save\mywine_clf.h5')
```

```
print('validata accuracy : %.4f' %new_model.evaluate(x_val, y_val)[1])
```

validata accuracy : 0.9585

전체 모델 불러와서
새 훈련없이 검증데이터 평가

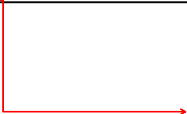
* keras **가중치** 모델 저장 / 복원

```
import numpy as np
import pandas as pd

winedf = pd.read_csv('wine.csv', header=None)
print(winedf.head())

winedata = winedf.values
x_data = winedata[:,0:12] # 훈련 특성데이터 추출
y_data = winedata[:,12] # 타겟 데이터 추출

# 훈련셋 , 검증셋 분리
from sklearn.model_selection import train_test_split
x_train, x_val, y_train, y_val = train_test_split(x_data,y_data,test_size=0.2, random_state=42)
```



모델 복원 후
검증데이터로 모델 평가

모델 설계

```
from tensorflow.keras.models import Sequential  
from tensorflow.keras.layers import Dense
```

```
def model_create():
```

```
    model = Sequential()  
    model.add(Dense(30, input_dim=12, activation='relu'))  
    model.add(Dense(12, activation='relu'))  
    model.add(Dense(8, activation='relu'))  
    model.add(Dense(1, activation='sigmoid'))
```

동일한 구조의
새 모델 설계를 위해
함수화

모델 컴파일

```
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])  
return model
```

```
model = model_create()
# 모델 훈련
model.fit(x_train, y_train, epochs=100, batch_size=200)

# 모델 평가
print('accuracy : %.4f ' %model.evaluate(x_train,y_train)[1])
```

```
# 모델 저장 == > 오류시 h5py 모듈 2.10.0 버전 설치
# 모델 저장 경로 : 영문 경로로 지정
model.save_weights('D:\Wkeras_model_save\save_weights\mywine_clf_weights.h5')
```

가중치 저장

```
del model    # 기존 모델 삭제

new_model = model_create()  # 동일 구조 새 모델 생성
```

```
# 저장된 모델 새로 불러옴
new_model.load_weights('D:\Wkeras_model_save\save_weights\mywine_clf_weights.h5')
print('validata accuracy : %.4f' %new_model.evaluate(x_val, y_val)[1])
```

새 모델에 저장된 가중치
로드해 적용

* 과대/과소 적합 파악을 위한 훈련셋/검증셋 시각화

fit() 메서드가 반환하는 결과를 저장하여
훈련셋 손실,정확도 와 검증셋 손실,정확도 출력

```
history = model.fit(x_train, y_train, validation_data=(x_val, y_val), epochs=10, batch_size=200)
```

훈련결과 반환(측정)값 history객체의 history dict에 저장

```
print(history.history.keys())
```

dict_keys(['val_loss', 'val_accuracy', 'loss', 'accuracy'])

검증데이터 손실

검증데이터 정확도

훈련데이터 손실

훈련데이터 정확도

```
print(history.history['loss'])  
print(history.history['accuracy'])  
print(history.history['val_loss'])  
print(history.history['val_accuracy'])
```

기본적으로 손실을 계산하지만
metrics 매개변수에 'accuracy' 추가로
매 epoch 마다 손실과 정확도 계산 저장

```
[0.5287141543875138, 0.39304731270174187, 0.3512934491535028, 0.3238498495121797, 0.30389539121836423]  
[0.81535417, 0.8589583, 0.8707917, 0.88189584, 0.88708335]  
[0.4673801041841507, 0.38554213058948517, 0.34939442535241444, 0.3470803085764249, 0.33978807347019513]  
[0.8370000123977661, 0.8615833520889282, 0.8722500205039978, 0.8711666464805603, 0.8775833249092102]
```

```
import numpy as np
import pandas as pd

winedf = pd.read_csv('wine.csv', header=None)
print(winedf.head())

winedata = winedf.values
x_data = winedata[:,0:12] # 훈련 특성데이터 추출
y_data = winedata[:,12] # 타겟 데이터 추출

# 훈련셋 , 검증셋 분리
from sklearn.model_selection import train_test_split
x_train, x_val, y_train, y_val = train_test_split(x_data,y_data,test_size=0.2, random_state=49)
```

모델 설계

```
from tensorflow.keras.models import Sequential  
from tensorflow.keras.layers import Dense
```

```
def model_create():  
    model = Sequential()  
    model.add(Dense(30, input_dim=12, activation='relu'))  
    model.add(Dense(12, activation='relu'))  
    model.add(Dense(8, activation='relu'))  
    model.add(Dense(1, activation='sigmoid'))
```

모델 컴파일

```
model.compile(loss='binary_crossentropy', optimizer='sgd', metrics=['accuracy'])  
return model
```

```
model = model_create()  
model.summary()
```

훈련셋, 검증셋 모델 훈련 결과 저장

```
history = model.fit(x_train, y_train, validation_data=(x_val, y_val), epochs=10, batch_size=200)
```

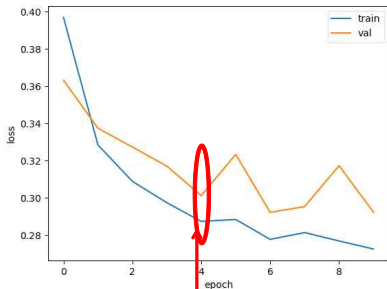
```
print(history.history.keys())
```

```
dict_keys(['loss', 'accuracy', 'val_loss', 'val_accuracy'])
```

```
print(history.history['loss'])    # 훈련셋 손실  
print(history.history['accuracy']) # 훈련셋 정확도  
print(history.history['val_loss']) # 검증셋 손실  
print(history.history['val_accuracy']) # 검증셋 정확도
```

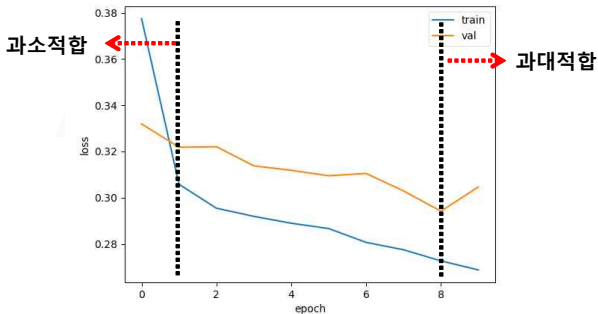
에포크에 따른 과대/과소 적합 파악을 위한 훈련셋/검증셋 시각화

```
import matplotlib.pyplot as plt  
plt.plot(history.history['loss']) # 훈련셋 손실  
plt.plot(history.history['val_loss']) # 검증셋 손실  
plt.xlabel('epoch')  
plt.ylabel('loss')  
plt.legend(['train', 'val'])  
plt.show()
```



에포크가 진행되면서 검증셋
손실이 다시 증가하는
과대 적합 문제

* 과대/과소 적합 해결 방법 (옵티마이저, dropout)



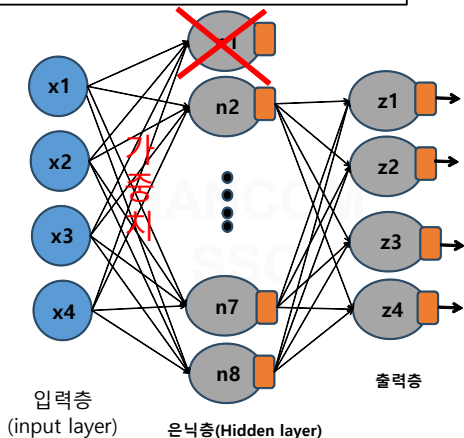
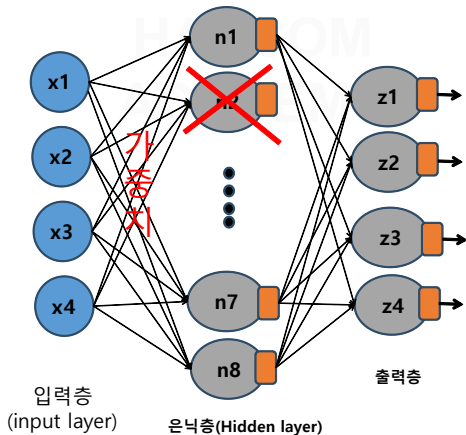
에포크가 진행되면서 훈련셋 손실은 지속적으로 감소하기 때문에 **검증셋 손실이 증가하는 시점을 최대한 늦추는 방법**과

검증셋 손실이 다시 증가하는 시점에서 **훈련 조기 종료 방법**을 적용!!

1. 방법) **optimizer** 하이퍼파라미터를 조정해서 과대적합을 완화시킬수 있는지 체크
앞의 예제에서 optimizer = 'sgd' 설정을 optimizer = 'adam' 으로 변경

2. 방법) 드롭아웃(dropout) 활용

: 훈련 과정에서 층에 있는 일부 뉴런을 랜덤하게 꺼서(**뉴런의 출력을 0 으로**) 과대 적합 방지
: 일부 뉴런이 랜덤하게 꺼지면 특정 뉴런에 과대하게 의존하지 않고 모든 입력에 주의를 기울여야 하기 때문에 과대 적합 방지 효과, 또한 여러개의 신경망이 앙상블된 효과



Model: "sequential"

Layer (type)	Output Shape	Param #
=====		
==		
dense (Dense)	(None, 30)	390
dense_1 (Dense)	(None, 12)	372
dropout (Dropout)	(None, 12)	0
dense_2 (Dense)	(None, 8)	104
dense_3 (Dense)	(None, 1)	9

dropout 층 추가
- 층처럼 사용되지만
 훈련되는 모델
 파라미터 없음
- 입력과 출력 크기
 동일

Total params: 875
Trainable params: 875
Non-trainable params: 0

* keras dropout 모델 적용 예

```
import numpy as np
import pandas as pd

winedf = pd.read_csv('wine.csv', header=None)
print(winedf.head())

winedata = winedf.values
x_data = winedata[:,0:12] # 훈련 특성데이터 추출
y_data = winedata[:,12] # 타겟 데이터 추출

# 훈련셋 , 검증셋 분리
from sklearn.model_selection import train_test_split
x_train, x_val, y_train, y_val = train_test_split(x_data,y_data,test_size=0.2, random_state=42)
```


모델 설계

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Dropout
```

```
def model_create(other_layer = None):
    model = Sequential()
    model.add(Dense(30, input_dim=12, activation='relu'))
    model.add(Dense(12, activation='relu'))
    if other_layer:
        model.add(other_layer)
    model.add(Dense(8, activation='relu'))
    model.add(Dense(1, activation='sigmoid'))
```

전달된 층이 있을 경우
두 층 사이에 새로운 층
추가 (Dropout 층)

모델 컴파일

```
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
return model
```

```
model = model_create( Dropout(0.3) ) →
model.summary()
```

Dropout 층 전달

드롭아웃 비율 :

보통 0.2(20%) ~ 0.5(50%) 사이를 사용

모델 훈련 결과 저장

```
history = model.fit(x_train, y_train, validation_data=(x_val, y_val), epochs=10, batch_size=200)
```

```
print('val accuracy : %.4f' % model.evaluate(x_val, y_val)[1])
```

```
print('y_train[0] : ', y_train[0]) # 실제값 : 레드와인
```

```
print('predict : ', model.predict(x_train[0:1])) # 레드와인으로 예측
```

에포크에 따른 과대/과소 적합 파악을 위한 훈련셋/검증셋 시각화

```
import matplotlib.pyplot as plt
```

```
plt.plot(history.history['loss']) # 훈련셋 손실
```

```
plt.plot(history.history['val_loss']) # 검증셋 손실
```

```
plt.xlabel('epoch')
```

```
plt.ylabel('loss')
```

```
plt.legend(['train', 'val'])
```

```
plt.show()
```

val accuracy : 0.9115

y_train[0] : 1.0

predict : [[0.9963981]]

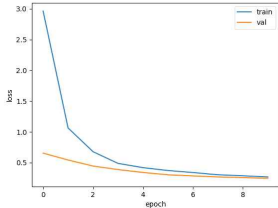
훈련이 끝난뒤 평가나 예측 단계에서는

드롭아웃을 사용하지 말아야 함

텐서플로와 케라스는 모델을 평가와 예측에

사용할때 자동으로 드롭아웃을 적용하지 않음으로

신경쓰지 않아도 됨!



* **keras - Callback(콜백) : 최상의 모델을 만들어 보자!!**
: 훈련 과정 중간에 어떤 작업을 수행할 있게 하는 객체
: **keras.callbacks** 패키지 아래에 있는 클래스

* 대표적으로 사용되는 callback ==> ModelCheckpoint , EarlyStopping

1. **ModelCheckpoint()** : 훈련 과정 중의 **모델을 저장할 때 사용되는 콜백함수**
: 매 에포크마다 모델을 저장하는 것은 비효율적이기 때문에
주로 최상의 검증 점수를 만들어 낼 때만 모델을 저장하는 방법으로 사용 

* 주요 파라미터 설명

filepath : 모델을 저장할 경로 입력, 문자열

monitor가 val_loss일때, 모델 경로를 **'{epoch:02d}-{val_loss:.5f}.h5'** 라고

입력하면, 에폭당 에폭에서의 val_loss.h5로 모델 저장, 예) 100-0.12892.h5 로 저장

monitor : 모델을 저장할 때, 기준이 되는 값을 지정,

validation set의 loss가 가장 적을 때 저장하고 싶으면 **'val_loss'** 입력

verbose : **1** 일 경우 ==> 저장됨을 화면에 표시, **0** 일 경우 ==> 화면표시 없이 모델 저장

save_best_only : **True** 일 경우 ==> monitor 되고 있는 값을 기준으로 가장 좋은 값으로 모델이 저장
False 일 경우 ==> 매 epoch마다 filepath로 저장

* keras callback - ModelCheckpoint 사용 예

```
import numpy as np
import pandas as pd

winedf = pd.read_csv('wine.csv', header=None)
print(winedf.head())

winedata = winedf.values
x_data = winedata[:,0:12] # 훈련 특성데이터 추출
y_data = winedata[:,12] # 타겟 데이터 추출

# 훈련셋 , 검증셋 분리
from sklearn.model_selection import train_test_split
x_train, x_val, y_train, y_val = train_test_split(x_data,y_data,test_size=0.2, random_state=42)
```

```
# 모델 설계
```

```
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import Dense
```

```
def model_create():
```

```
    model = Sequential()
```

```
    model.add(Dense(30, input_dim=12, activation='relu'))
```

```
    model.add(Dense(12, activation='relu'))
```

```
    model.add(Dense(8, activation='relu'))
```

```
    model.add(Dense(1, activation='sigmoid'))
```

```
# 모델 컴파일
```

```
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
```

```
return model
```

```
model = model_create()
```

```
model.summary()
```

```
from tensorflow.keras.callbacks import ModelCheckpoint
```

```
modelpath = 'D:\Wkeras_model_save\model\W{epoch:02d}-{val_loss:.5f}.h5'
```

```
checkpointer = ModelCheckpoint(filepath=modelpath, monitor='val_loss', verbose=1,  
                               save_best_only=True)
```

```
model.fit(x_train, y_train, validation_data=(x_val,y_val), epochs=100, batch_size=200,  
         verbose=0, callbacks=[checkpointer])
```



Epoch 00097: val_loss improved from 0.08959 to 0.08845, saving model to D:\Wkeras_model_save\model\W97-0.08845.h5

Epoch 00098: val_loss **did not improve** from 0.08845

Epoch 00099: val_loss did not improve from 0.08845

Epoch 00100: val_loss did not improve from 0.08845

val_loss 값이 개선될 때만 모델 저장



이후 해당 모델을 load_model()로 모델
읽어와서 검증 평가 및 예측 수행

2. EarlyStopping() : 모델 loss 개선이 없을 경우 훈련 진행 중 훈련을 미리 종료시키는 조기 종료 콜백

- 훈련 epoch(에포크) 제한으로 컴퓨터 자원 과 시간을 아낄 수 있으며 무엇보다 에포크가 지속되면서 검증(테스트) 점수가 더 이상 개선되지 않고 다시 증가하는 **과대적합을 막아주는 효과**
- ModelCheckpoint 콜백과 병행해 사용하면 가장 낮은 검증 손실의 모델을 파일로 저장할 수 있고 검증 손실이 다시 증가할 때(과대적합 발생) 훈련을 중지할 수 있음!!

* 주요 파라미터 설명

monitor : EarlyStopping의 기준이 되는 값을 지정,
'val_loss' 입력하면 val_loss가 더 이상 감소하지 않을 경우 조기 종료

patience : 얼마나 조기종료 되는것을 참을지 지정, 예) patience = 3 으로 지정시 3번 연속 검증 점수가 향상되지 않을 경우 훈련 종료 , 보통 restore_best_weights = True 로 같이 사용

verbose : 1 일 경우 ==> 화면에 적용 표시, 0 일 경우 ==> 화면표시 없이 종료

restore_best_weights : True 일 경우
==> 훈련 중지 후, model의 weight를 monitor하고 있던 값중 가장 좋은 값으로 모델 weight를 자동 복원
False 일 경우 ==> 마지막 훈련이 끝난 후의 weight로 놔둠

*** 조기종료 콜백 사용 전 적절한 epoch(에포크) 체크를 위한
훈련 손실 과 검증 손실 시각화**

~~ 훈련셋, 검증셋 분리 , 모델 설계 부분 이전 코드와 동일 ~~

```
history = model.fit(x_train, y_train, validation_data=(x_val,y_val), epochs=1000, batch_size=300,  
verbose=0, callbacks=[checkpointer])
```

```
val_loss = history.history['val_loss']  
train_loss = history.history['loss']
```

검증 손실, 훈련 손실
데이터 참조

epochs : 1000
batch_size : 300
으로 수정 테스트 진행

```
import matplotlib.pyplot as plt
```

```
x_len = np.arange(len(train_loss))  
plt.plot(x_len, val_loss, c = 'red')  
plt.plot(x_len, train_loss, c = 'blue')
```

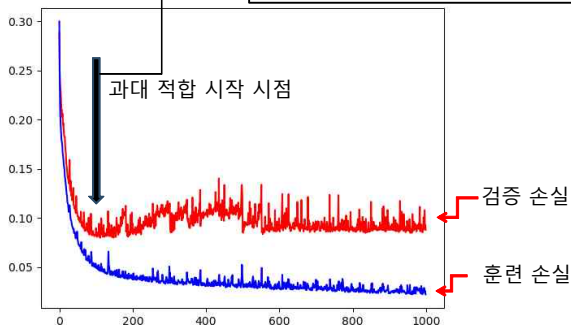
검증 손실 : 빨간선 그래프

훈련 손실 : 파랑선 그래프

```
plt.show()
```


*** 조기종료 콜백 사용 전 적절한 epoch(에포크) 체크를 위한
훈련 손실 과 검증 손실 시각화**

결과 그래프를 통해 훈련회수(epochs)가 진행될 수록 훈련 정확도는 증가하지만, 약 100 epochs 이후에는 검증 손실이 개선되지 않고 다시 증가하는 과대적합 현상 발생됨을 알수 있음



결론, 훈련회수(epochs)가 너무 작아서도 안되며 너무 많으면 과대적합 문제가 발생됨으로 적절한 훈련회수(epochs)를 결정해야함!



이때!!

훈련(epochs)이 진행되는 도중 검증 손실이 더 이상 개선되지 않은 시점에 훈련 조기 종료 되게하면 **안심하고 에포크 회수를 크게 잡을 수 있는 에포크 회수 결정 문제점 해결!!**

* keras callback - EarlyStopping(조기종료) 사용 예

~~ 훈련셋, 검증셋 분리 이전 코드 동일 ~~

#모델 설계

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

```
def model_create():
```

```
    model = Sequential()
    model.add(Dense(30, input_dim=12, activation='relu'))
    model.add(Dense(12, activation='relu'))
    model.add(Dense(8, activation='relu'))
    model.add(Dense(1, activation='sigmoid'))
```

모델 컴파일

```
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
return model
```

```
model = model_create()
model.summary()
```

```
from tensorflow.keras.callbacks import ModelCheckpoint
from tensorflow.keras.callbacks import EarlyStopping
```

조기 종료 콜백 추가

```
modelpath = 'D:\k\keras_model_save\model\BestModel.h5'
checkpointer = ModelCheckpoint(filepath=modelpath, monitor='val_loss',verbose=1,
                               save_best_only=True)
earlystop_cb = EarlyStopping(monitor='val_loss', patience=3,restore_best_weights=True)

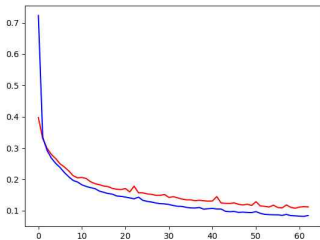
history = model.fit(x_train, y_train, validation_data=(x_val,y_val),epochs=1000, batch_size=300,
                    verbose=0, callbacks=[checkpointer, earlystop_cb])
```

```
val_loss = history.history['val_loss']
train_loss = history.history['loss']
```

```
import matplotlib.pyplot as plt
```

```
plt.plot(val_loss, c = 'red')
plt.plot(train_loss, c = 'blue')
```

```
plt.show()
```



Thank You